

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №4
по дисциплине «Построение и анализ алгоритмов»
Тема: Поиск подстроки в строке

Студент гр. 4345

Гавриш М.А.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2026

Цель работы

Изучить и реализовать алгоритм Кнута-Морриса-Пратта для поиска подстроки в строке.

Задание

1. Реализуйте алгоритм КМП и с его помощью для заданных шаблона P ($|P| \leq 2500$) и текста T ($|T| \leq 5000000$) найдите все вхождения P в T .

Вход:

- Первая строка – P
- Вторая строка – T

Выход:

Индексы начал вхождений P в T , разделённые запятой; если P не входит в T , то вывести -1.

2. Заданы две строки A ($|A| \leq 5000000$) и B ($|B| \leq 5000000$).

Определить, является ли A циклическим сдвигом B (это значит, что A и B имеют одинаковую длину и A состоит из суффикса B , склеенного с префиксом B). Например, defabc является циклическим сдвигом abcdef.

Вход:

- Первая строка – A
- Вторая строка – B

Выход:

Если A является циклическим сдвигом B , то индекс начала строки B в A ; иначе вывести -1. Если возможно несколько сдвигов, вывести первый индекс.

Основные теоретические положения

Алгоритм Кнута–Морриса–Пратта (КМП) — классический метод точного поиска подстроки в тексте, отличающийся линейной временной сложностью. В отличие от наивного подхода, который при каждом несовпадении сдвигает шаблон на одну позицию и начинает сравнение заново, КМП использует предварительно вычисленную информацию о внутренней структуре самого шаблона. Это позволяет пропускать заведомо неперспективные сдвиги, избегая повторного чтения уже обработанных символов текста. Теоретический предел эффективности поиска подстроки составляет $O(|T| + |P|)$, и алгоритм КМП достигает этой границы.

Ключевым элементом алгоритма является префикс-функция π . Для строки-шаблона P длины m значение $\pi[i]$ равно длине наибольшего собственного префикса подстроки $P[0...i]$, который одновременно выступает её суффиксом. Массив π вычисляется за $O(m)$ и служит «картой откатов»: при рассогласовании символов алгоритм не возвращается к началу шаблона, а переносит указатель сравнения на позицию $\pi[j-1]$, где j — количество уже совпавших символов.

Выполнение работы

Для решения поставленных задач была реализована программа на языке C++. В ходе работы рассмотрены два применения алгоритма КМП.

Часть 1. Поиск всех вхождений шаблона в текст

Программа считывает строку-шаблон P и исходный текст T . Основной вычислительный блок разделён на логические этапы: построение таблицы префикс-функции и непосредственный поиск.

В коде используются следующие ключевые переменные и конструкции:

- `vector<int> pi(m, 0)` — массив, хранящий значения префикс-функции для шаблона. Позволяет определять длину наибольшего совпадающего префикса-суффикса на каждом шаге, избегая перебора.
- `int j` — счётчик совпавших символов шаблона. Изначально равен 0, увеличивается при $T[i] = P[j]$ и корректируется через $pi[j-1]$ при несовпадении.
- `int i` — индекс текущего символа в тексте T . Изменяется только в сторону увеличения, что гарантирует однократный проход по тексту без возвратов указателя.
- `vector<int> positions` — контейнер для сохранения начальных индексов найденных вхождений. Используется `reserve()` для предотвращения лишних аллокаций памяти при росте вектора.

Логика поиска построена на цикле по тексту. При каждом шаге происходит проверка $T[i] = P[j]$. Если символы различаются, значение j корректируется по таблице π до тех пор, пока не будет найдено возможное продолжение совпадения или пока j не станет равным 0. При достижении $j = m$ (полное совпадение шаблона) индекс начала вхождения ($i - m + 1$) записывается в массив результатов, а j сбрасывается в $pi[m-1]$ для корректного поиска перекрывающихся вхождений. Если массив результатов остаётся пустым, программа выводит -1.

Часть 2. Проверка на циклический сдвиг

Вторая задача сводится к проверке, можно ли получить строку A циклическим сдвигом строки B . Математически это эквивалентно утверждению: A является циклическим сдвигом B тогда и только тогда, когда B встречается как подстрока в конкатенации $A + A$ (при условии равенства длин).

В реализации сначала проверяется равенство длин входных строк. Затем формируется вспомогательная строка $S = A + A$. Далее запускается модифицированная версия КМП, где шаблон — это B , а текст — S . Поиск останавливается сразу после первого найденного вхождения, так как по условию требуется вывести минимальный индекс сдвига. Найденная позиция вычисляется как $i - m + 1$. Если цикл завершается без совпадений, выводится -1.

Данный подход позволяет переиспользовать уже написанный механизм КМП, добавляя лишь проверку длин и формирование удвоенной строки. Сложность остаётся линейной, а память расходуется пропорционально суммарному размеру входных данных.

Оценка сложности

- Временная сложность: $O(|T| + |P|)$. Построение префикс-функции требует $O(|P|)$ операций, поиск по тексту — $O(|T|)$. Каждый символ участвует в константном числе сравнений благодаря свойству монотонности откатов: суммарное количество уменьшений j не может превысить суммарное количество его увеличений.
- Пространственная сложность: $O(|P|)$ для хранения массива π и $O(|T|)$ для хранения входных данных. В первой задаче дополнительная память пропорциональна количеству найденных вхождений, во второй — ограничена размером конкатенированной строки S .

Исходный код программ приведён в приложении А. Результаты тестирования корректности и производительности оформлены в таблицах 1 и 2.

Таблица 1 – Результаты проверки алгоритма поиска подстроки

Ввод	Результат	Комментарий
ab abab	0,2	Пример из условия задачи
defabc abcdef	-1	Пример из условия задачи
zz pizzzzza	2,3,4,5	Несколько найденных подстрок
za pizza	3	Одна найденная подстрока

Таблица 2 – Результаты проверки алгоритма определения циклического сдвига

Ввод	Результат	Комментарий
ab abab	-1	Пример из условия задачи
defabc abcdef	3	Пример из условия задачи
pizza zapiz	3	Найден циклический сдвиг

Выводы

В рамках лабораторной работы был изучен алгоритм Кнута–Морриса–Пратта. На практике реализованы две задачи: поиск всех вхождений подстроки и определение циклического сдвига строк.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Файл lab4_1.cpp:

```
#include <iostream>
#include <string>
#include <vector>

using namespace std;

int main() {
    string p, t;
    cout << "Введите строку p: ";
    cin >> p;
    cout << "Введите строку t: ";
    cin >> t;

    int n = t.length();
    int m = p.length();

    vector<int> pi(m, 0);
    for (int i = 1; i < m; i++) {
        int j = pi[i - 1];
        while (j > 0 && p[i] != p[j])
            j = pi[j - 1];
        if (p[i] == p[j])
            j++;
        pi[i] = j;
    }

    cout << "\nЗначения префикс-функции для \"" << p << "\": ";
    for (size_t k = 0; k < pi.size(); ++k) {
        cout << pi[k];
        if (k != pi.size() - 1) cout << " ";
    }
    cout << "\n\n";

    vector<int> positions;
    int i = 0;
    int j = 0;

    while (i < n) {
        if (t[i] == p[j]) {
            cout << "t[" << i << "] = " << t[i] << " и p[" << j << "]
= " << p[j] << " совпали\n";
            i++;
            j++;
            cout << "Идем вперед по префикс-функции, новый j: " << j
<< "; i = " << i << "\n";

            if (j == m) {
                cout << "Подстрока " << p << " найдена на индексе " <<
(i - j) << "\n";
                positions.push_back(i - j);
            }
        }
    }
}
```

```

        j = pi[j - 1];
        cout << "(Успех) Идем назад по префикс-функции, новый
j: " << j << "\n";
    }
    } else {
        cout << "t[" << i << "] = " << t[i] << " и p[" << j << "]
= " << p[j] << " не совпали\n";
        if (j > 0) {
            j = pi[j - 1];
            cout << "Идем назад по префикс-функции, новый j: " <<
j << "\n";
        } else {
            i++;
            cout << "Следующий i = " << i << "\n";
        }
    }
}

cout << "\nРезультат: ";
if (positions.empty()) {
    cout << "-1";
} else {
    for (size_t k = 0; k < positions.size(); k++) {
        if (k > 0) cout << ",";
        cout << positions[k];
    }
}
cout << "\n";

return 0;
}

```

Файл lab4_2.cpp:

```

#include <iostream>
#include <string>
#include <vector>

using namespace std;

vector<int> compute_pi(const string& P) {
    int m = P.length();
    vector<int> pi(m, 0);
    for (int i = 1; i < m; i++) {
        int j = pi[i - 1];
        while (j > 0 && P[i] != P[j])
            j = pi[j - 1];
        if (P[i] == P[j])
            j++;
        pi[i] = j;
    }
    return pi;
}

int main() {
    string A, B;
    cout << "Введите строку A: ";
}

```

```

cin >> A;
cout << "Введите строку B: ";
cin >> B;

if (A.length() != B.length()) {
    cout << "\nДлины строк не совпадают. Циклический сдвиг
невозможен.\n";
    cout << "\nРезультат: " << -1;
    return 0;
}
if (A.empty()) {
    cout << "\nРезультат: " << 0;
    return 0;
}

string S = A + A;
int n = S.length();
int m = B.length();

vector<int> pi = compute_pi(B);

cout << "\nЗначения префикс-функции для \"" << B << "\": ";
for (size_t k = 0; k < pi.size(); ++k) {
    cout << pi[k];
    if (k != pi.size() - 1) cout << " ";
}
cout << "\n\n";

int i = 0;
int j = 0;

while (i < n) {
    if (S[i] == B[j]) {
        cout << "S[" << i << "] = " << S[i] << " и B[" << j << "]
= " << B[j] << " совпали\n";
        i++;
        j++;
        cout << "Идем вперед по префикс-функции, новый j: " << j
<< "; i = " << i << "\n";

        if (j == m) {
            int shift_idx = i - j;
            cout << "Найдена полная подстрока \"" << B << "\" в S
на индексе " << shift_idx << "\n";
            cout << "\nРезультат: " << shift_idx << "\n";
            return 0;
        }
    } else {
        cout << "S[" << i << "] = " << S[i] << " и B[" << j << "]
= " << B[j] << " не совпали\n";
        if (j > 0) {
            j = pi[j - 1];
            cout << "Идем назад по префикс-функции, новый j: " <<
j << "\n";
        } else {
            i++;
            cout << "Увеличиваем i = " << i << "\n";
        }
    }
}

```

```
    }  
}  
  
cout << "\nРезультат: " << -1;  
return 0;  
}
```